DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Seguridad web y HTTPS

Tema 3 — Publicación y ejecución de aplicaciones web



Objetivo de la sesión

Punto de partida

Aplicaciones HTTP detrás de un proxy.

Imágenes ya construidas y configuración de despliegue.

Qué incorporamos

Control de acceso con Basic.

HTTPS, certificados y confianza.

Emisión, renovación y cabeceras de seguridad.

Nginx concentra la entrada pública y la protección del transporte.

Del laboratorio al servidor remoto

Preparación de la imagen

Construcción y pruebas antes del despliegue.

Publicación en un registro accesible.

La imagen incorpora el entorno de ejecución.

Ejecución en el host

Motor de contenedores y Compose.

Configuración y secretos.

Descarga de imágenes y ejecución sin recompilar.

El host necesita un entorno compatible con las imágenes publicadas.

Publicación de puertos

```
# Publicación en las interfaces del host
ports:
  - "80:80"

# Prueba accesible solo desde el propio host
ports:
  - "127.0.0.1:8080:80"
```

La exposición depende también de las reglas de red y del cortafuegos.

Nginx publica los puertos web. Las aplicaciones y la base de datos permanecen internas.

Nombres públicos y selección del sitio

DNS

El nombre apunta a la IP pública correcta.

Un cambio de IP puede exigir actualizar el DNS.

El nombre debe resolver desde Internet.

Nginx

SNI permite elegir el certificado durante TLS.

Host permite seleccionar el sitio HTTP.

El certificado debe cubrir el nombre solicitado.

Los nombres .test de los ejemplos se sustituyen por nombres públicos en la práctica.

Basic Authentication

Intercambio

Sin credenciales, Nginx responde 401.

WWW-Authenticate solicita autenticación.

El cliente reenvía con Authorization.

Credenciales

Authorization: Basic <valor>

El valor codifica usuario:contraseña en Base64.

La codificación es reversible.

Con credenciales aceptadas, la petición continúa hacia el contenido o la aplicación.

Protección de una ruta

```
location /privado/ {
    auth_basic "Zona restringida";
    auth_basic_user_file
        /etc/nginx/auth/usuarios.htpasswd;

    # Aquí se configura el contenido o proxy.
}
```

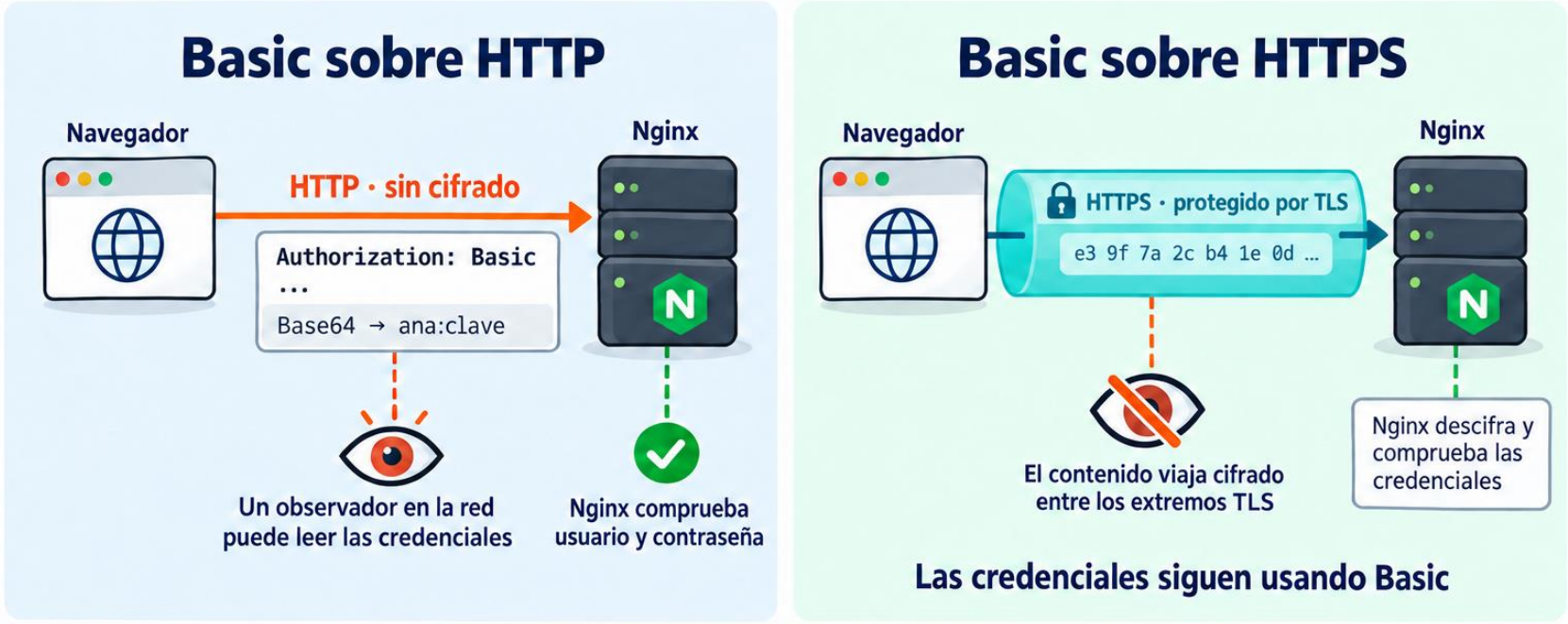
El fichero contiene hashes y sigue siendo sensible. Se mantiene fuera del repositorio.

Estas directivas se añaden al location que ya sirve contenido o hace de proxy.

Acceso y protección del transporte

Controlar el acceso no es cifrar el tráfico

Basic Authentication y HTTPS resuelven problemas distintos



Basic controla el acceso. TLS protege el transporte.
Base64 es una codificación reversible, no un cifrado.

Ejemplo didáctico: ana:clave no es una contraseña recomendada.

Qué aporta TLS

Garantías

Confidencialidad del contenido.

Detección de modificaciones del tráfico.

Autenticación del servidor.

Límites

Persisten los errores de programación y autorización.

El servidor puede estar comprometido.

Algunos metadatos, como la IP de destino, siguen siendo visibles.

HTTPS protege la comunicación entre los extremos de la conexión TLS.

Establecimiento de HTTPS



Certificados y cadena de confianza

Certificado público, clave privada y confianza

El certificado se comparte; la clave privada se protege

Certificado público



- Nombres válidos (SAN)
- Clave pública
- Fechas de validez
- Emisor
- Firma de la CA

Puede incluir varios nombres: **web.ejemplo.test** y **docs.ejemplo.test**

Clave privada



- Permanece en el servidor
- No se envía al navegador
- Permite demostrar la identidad del servidor

No forma parte del certificado público

Cadena de confianza



Un certificado válido acredita el nombre del servidor, no que toda la aplicación sea segura.

Nombres válidos y confianza

SAN y cadena

SAN enumera los nombres cubiertos.

El servidor presenta el certificado y los intermedios necesarios.

La raíz confiable está en el cliente.

Autofirmado y CA pública

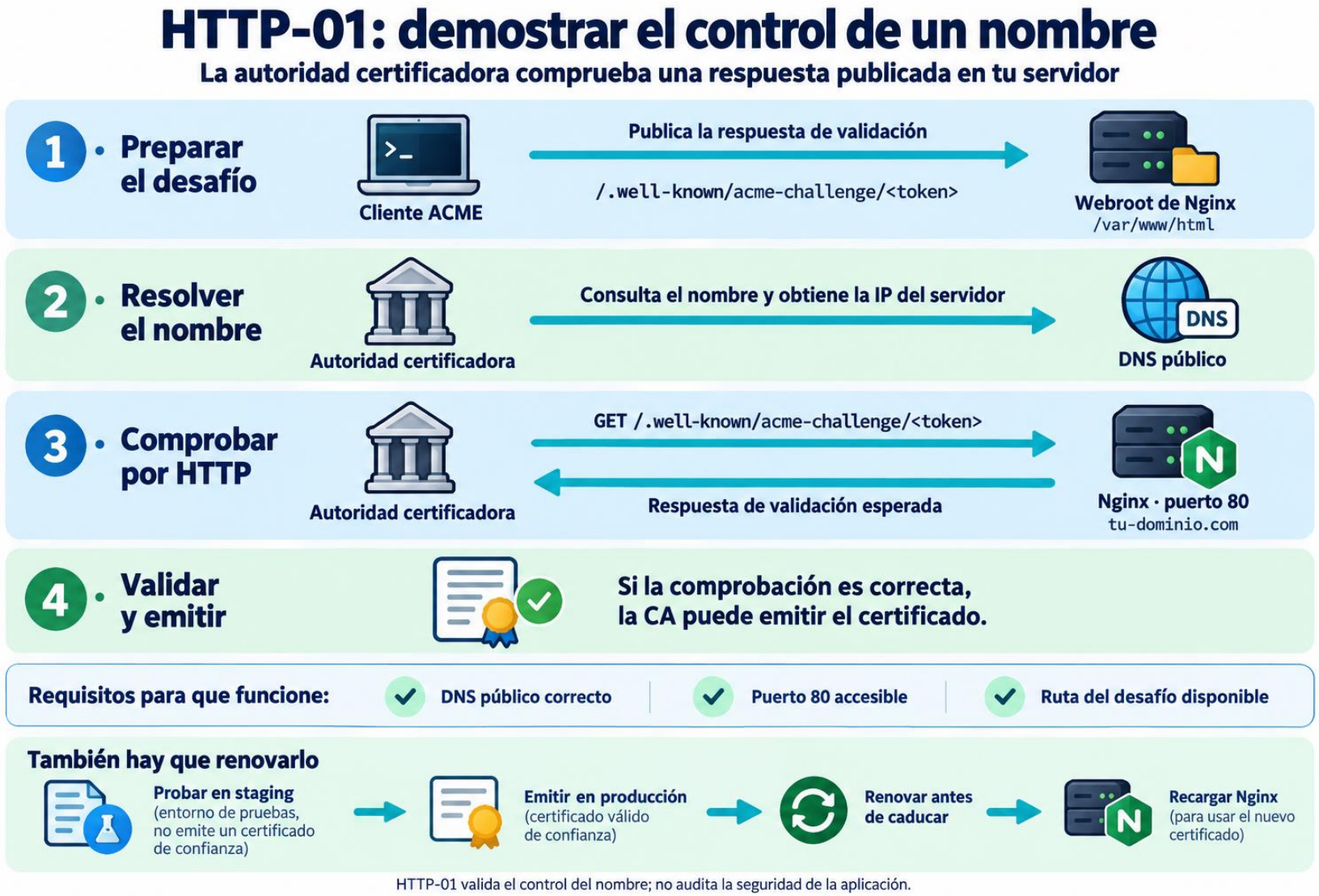
Un autofirmado requiere gestionar la confianza del cliente.

Una CA pública reconocida permite validar sin instalar confianza adicional.

Nombre y vigencia también deben ser correctos.

La clave privada queda fuera de los repositorios y de las imágenes públicas.

Validación del nombre con HTTP-01



Condiciones para HTTP-01

Acceso desde Internet

DNS público dirigido al servidor.

Puerto 80 accesible.

Ruta del desafío sin autenticación Basic.

Fichero de validación

El cliente ACME escribe donde Nginx lee.

Webroot elegido: /var/www/acme.

HTTP-01 no permite certificados comodín.

La CA valida el control del nombre, sin auditar la seguridad de la aplicación.

Pruebas, emisión y renovación

Antes de emitir

Probar el recorrido en staging.

Sus certificados no tienen confianza pública habitual.

Solicitar producción cuando la validación funcione.

Durante el servicio

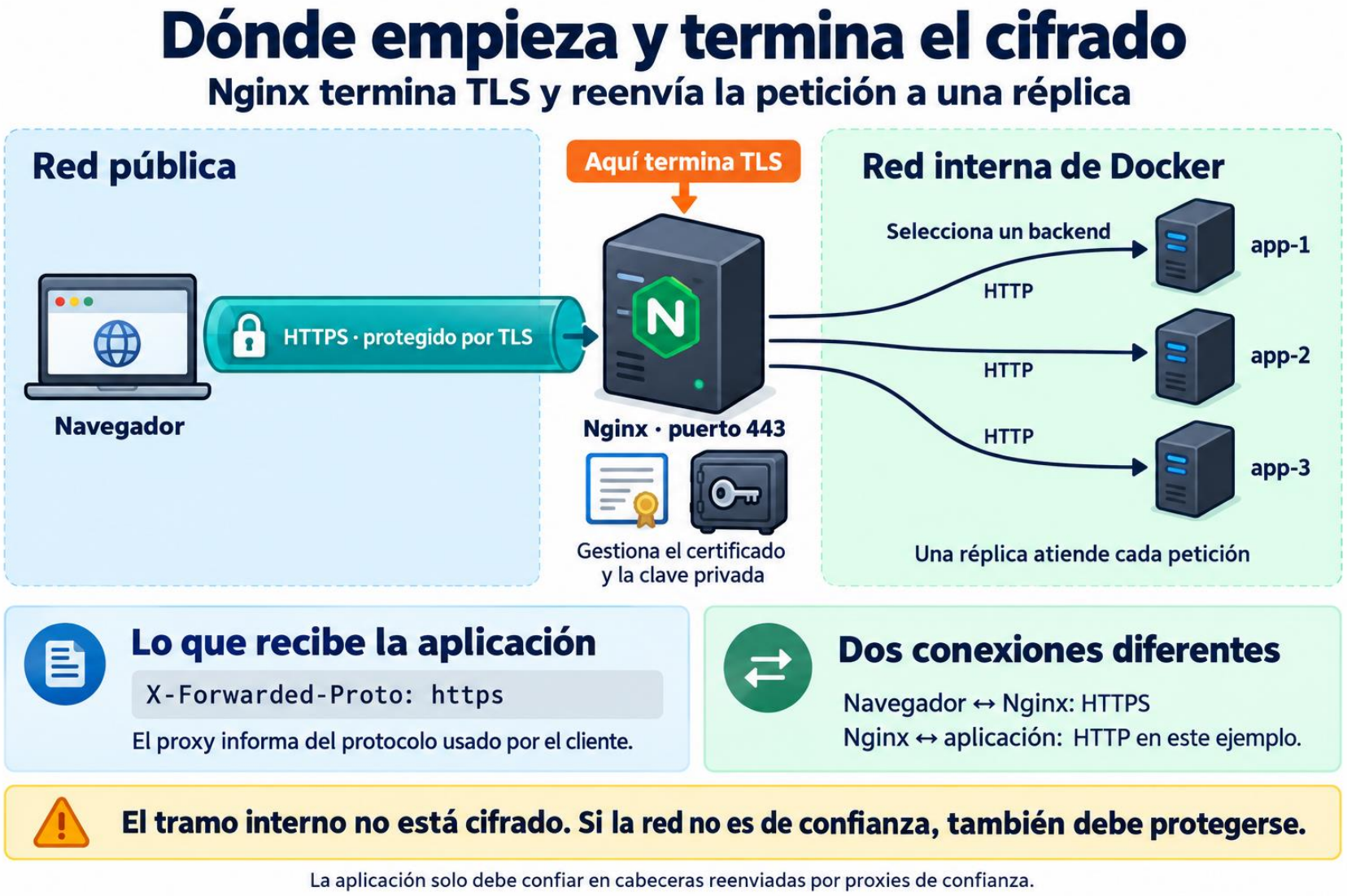
Renovar antes de caducar.

Persistir certificados y claves.

Comprobar configuración, recargar y verificar el certificado servido.

Tras renovar, Nginx debe recargarse para utilizar los nuevos ficheros.

Terminación TLS en el proxy



Bloque HTTPS en Nginx

```
server {
    listen 443 ssl;
    server_name web.ejemplo.test;

    ssl_certificate     /etc/letsencrypt/live/mi-cert/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/mi-cert/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    # Aquí se configura el contenido o proxy.
}
```

fullchain.pem incluye el certificado y los intermedios. privkey.pem contiene la clave privada.

Este fragmento presupone certificados emitidos y ficheros accesibles para Nginx.

Protocolo original y cabeceras

```
proxy_set_header X-Forwarded-Proto $scheme;
```

La directiva se integra con Host y las demás cabeceras del proxy.

Si el cliente llega por HTTPS, el valor enviado es https, aunque el backend reciba HTTP.

La aplicación debe confiar únicamente en los proxies previstos.

Redirección y política HSTS



Puerto 80: desafío y redirección

```
server {
    listen 80;
    server_name web.ejemplo.test;
    location ^~ /.well-known/acme-challenge/ {
        root /var/www/acme;
        auth_basic off;
        try_files $uri =404;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}
```

El ejemplo conserva la ruta ACME y redirige el resto a HTTPS.

Cómo aprende HSTS el navegador

Aprendizaje

Recibe la cabecera por HTTPS válido.

Conserva la política durante el periodo indicado.

Otra respuesta con HSTS actualiza ese periodo.

Aplicación

Usa HTTPS antes de conectar mientras la política siga vigente.

Ignora HSTS recibido por HTTP.

HSTS no hace válido un certificado incorrecto.

En el laboratorio usamos max-age=300, sin includeSubDomains ni preload.

Cabeceras del bloque HTTPS

```
add_header Strict-Transport-Security "max-age=300" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy
    "strict-origin-when-cross-origin" always;
```

HSTS: uso de HTTPS.
nosniff: respeto del tipo MIME.
Referrer-Policy: información enviada en Referer.

always permite añadirlas también a respuestas de error.

Conviene verificar las cabeceras reales que recibe el navegador.

Seguridad en varias capas

Transporte y acceso

TLS protege la comunicación.

La autenticación identifica al usuario.

La autorización decide qué puede hacer.

Aplicación e infraestructura

Validación y programación segura.

Permisos y protección de secretos.

Actualizaciones y control de puertos.

Cada capa conserva responsabilidades propias.

Cierre de la sesión

Qué debemos poder explicar y comprobar

Qué expone el despliegue y qué permanece interno.

Cómo se complementan Basic Authentication y TLS.

Qué valida el cliente al recibir un certificado.

Qué necesita HTTP-01 para emitir y renovar.

Dónde termina TLS y cómo actúan redirección y HSTS.

En la actividad, Nginx será la entrada pública con HTTPS y control de acceso.